科隆威声光报警器连接使用说明

USB 可控声光报警器连接使用说明 2
1. 产品简介 2
2. 产品连接 2
2.1 连接智能灯 usb，使用设备管理器查询连接端口号 2
2.2 使用本公司提供的串口连接工具连接设备端口号 2
3 使用软件编程控制信号灯（我们有多种代码，Java，python，VB 等等） 4

USB/485 可控声光报警器连接使用说明

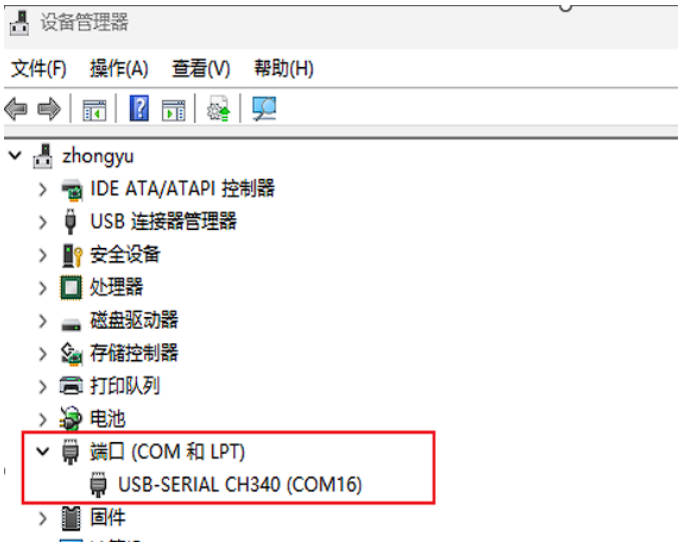
1. 产品简介

本公司推出的声光报警器，具有低功耗、长寿命、安装灵活、方便等特点；本产品支持 modbus RTU 标准协议进行连接，可以根据需求以不同的灯光颜色以及闪灯方式搭配蜂鸣报警来达到警示报警等功能，比如：红色的灯光可以用于在声音报警和闪光报警，只要触发即发出闪光信号和蜂鸣报警信号，让现场人员迅速知道现场已发生事故或设备故障提示，尽快采取相应措施，避免产生重大人员、财物损失；通过智能控制模块还可 接入分布智能火灾自动报警控制系统中。 本产品可应用 5V USB 电压或 10-30V 480 串口工作的火灾报警控制系统、安防监控报警系统及 其他报警系统中。

2. 产品连接

2.1 连接智能灯串口，使用设备管理器查询连接串口号

USB 串口是带有 CH340 标识的串口（无法识别串口可能是没有相应驱动，我们提供驱动）



2.2 使用本公司提供的串口连接工具连接设备端口号

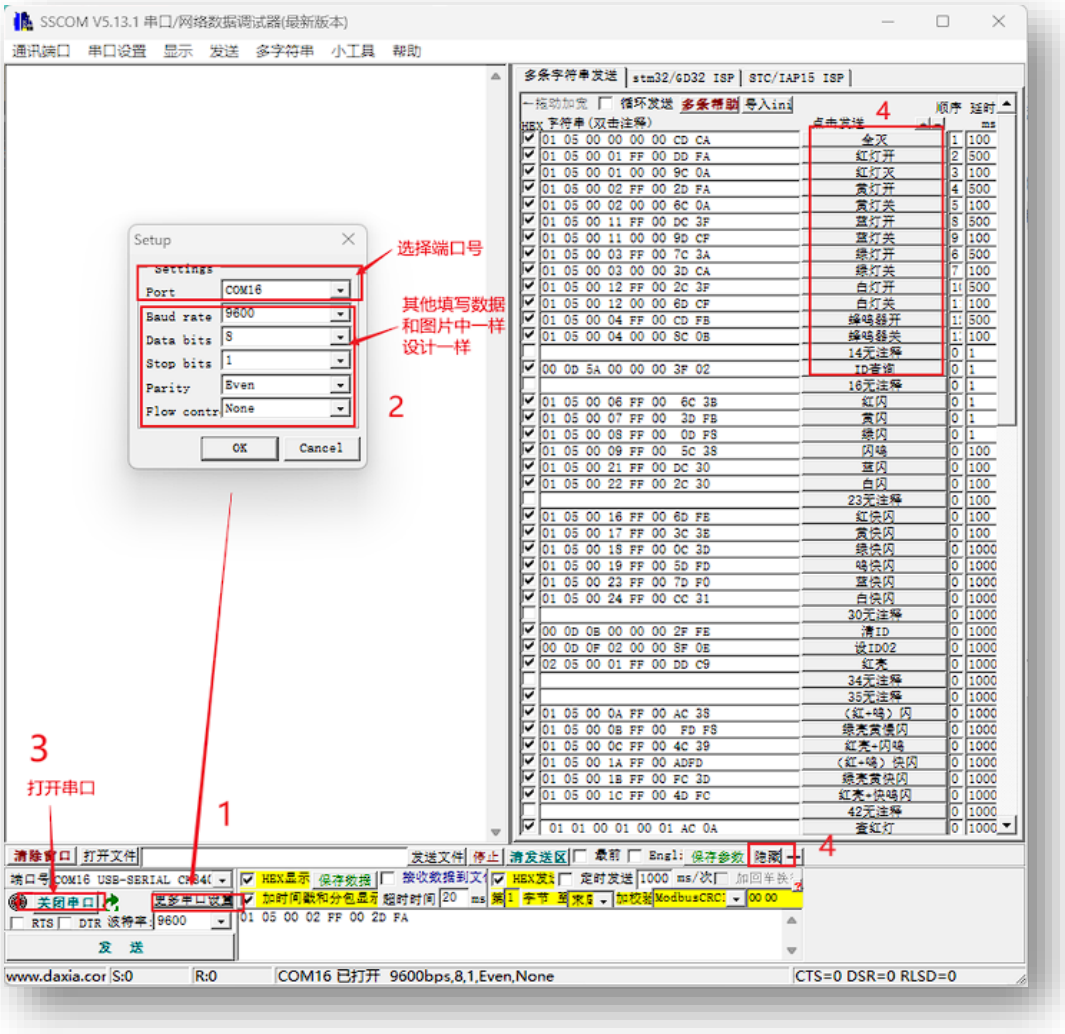
打开串口工具参照以下图片配置连接参数连接信号灯

科隆威声光报警器连接使用说明

1 第一步：选择更多串口配置;

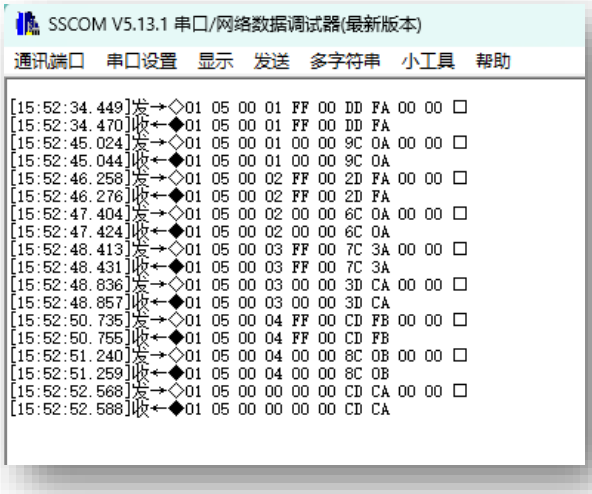
2.第二步：选择对应的串口，设置波特率为 9600，parity 规则选择 Even

3.第三步：打开串口（点击后变为关闭串口）



4.第四步：点击图中位置的扩展（点击后变为隐藏）按钮发送指令

如果连接成功发送指令会出现收发命令提示



所有控制命令

可利用不同的命令组合形成跟多报警效果，但要求单个命令必须间隔最少 50ms 才能有效

字符串命令	功能
01 05 00 00 00 00 CD CA	全灭
01 05 00 01 FF 00 DD FA	红灯开
01 05 00 01 00 00 9C 0A	红灯灭
01 05 00 02 FF 00 2D FA	黄灯开
01 05 00 02 00 00 6C 0A	黄灯灭
01 05 00 03 FF 00 7C 3A	绿灯开
01 05 00 03 00 00 3D CA	绿灯灭
01 05 00 04 FF 00 CD FB	蜂鸣开
01 05 00 04 00 00 8C 0B	蜂鸣关

3 使用软件编程控制信号灯（我们有多种代码，Java，python，VB 等等）

现在以 java 代码控制程序为例

首先引入 java 窗口连接工具包，字节输入输出流

```
import com.fazecast.jSerialComm.*;
import java.io.IOException;
import java.io.OutputStream;
```

```
Import    com.fazecast.jSerialComm.*;
import    java.io.IOException;
import    java.io.OutputStream;
```

直接使用 Java8 原生包

预定义的十六进制命令（字符串形式）

```
public static final String COMMAND_CLOSE_ALL = "0105000000000CDCA"; // 关闭所有灯光  2个用法
public static final String COMMAND_GREEN = "01050003FF007C3A";    // 绿灯亮  1个用法
public static final String COMMAND_YELLOW = "0105000200006C0A";    // 黄灯亮  1个用法
public static final String COMMAND_RED = "01050001FF00DDFA";      // 红灯亮  1个用法
```

```
public static final String COMMAND_CLOSE_ALL = "0105000000000CDCA"; // 关闭所有灯光
public static final String COMMAND_GREEN = "01050003FF007C3A";      // 绿灯亮
public static final String COMMAND_YELLOW = "0105000200006C0A";     // 黄灯亮
public static final String COMMAND_RED = "01050001FF00DDFA";       // 红灯亮
...
```

编辑*转换十六进制字符串为字节数组*工具



```
/**
 * 转换十六进制字符串为字节数组
 */
public static final class HexUtils {
    private static final int[] DEC = new int[]{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, -1, -1, -1, -1, -1, -1, -1, 10, 11,
            12, 13, 14, 15, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,
            -1, -1, -1, -1, -1, -1, -1, 10, 11, 12, 13, 14, 15};
    private static final byte[] HEX = new byte[]{48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 97, 98, 99, 100, 101, 102};
    private static final char[] hex = {48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 97, 98, 99, 100, 101, 102};

    public HexUtils() {
    }

    /**
     * 字符获取十进制
     *
     * @param index
     * @return
     */
    public static int getDec(int index) {
        try {
            return DEC[index - 48];
        } catch (ArrayIndexOutOfBoundsException var2) {
            return -1;
        }
    }

    public static byte[] fromHexString(String input) {
        if (input == null) {
            return null;
        } else if ((input.length() & 1) == 1) {
            // 肯定是偶数位，奇数位转换失败：一个byte对应两个字符
            throw new IllegalArgumentException("hexUtils.fromHex.oddDigits");
        } else {
            char[] inputChars = input.toCharArray();
            byte[] result = new byte[input.length() >> 1];
            for (int i = 0; i < result.length; ++i) {
                int upperNibble = getDec(inputChars[2 * i]);
                int lowerNibble = getDec(inputChars[2 * i + 1]);
                if (upperNibble < 0 || lowerNibble < 0) {
                    // 字符不存在
                    throw new IllegalArgumentException("hexUtils.fromHex.nonHex");
                }

                result[i] = (byte) ((upperNibble << 4) + lowerNibble);
            }

            return result;
        }
    }
}
```

/**

*** 转换十六进制字符串为字节数组**

*/

public static final class HexUtils {

private static final int[] *DEC* = new int[]{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, -1, -1, -1, -1, -1,

-1,

-1, -1, 10, 11, 12, 13, 14, 15, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -

```
1,
        -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, 10, 11, 12, 13, 14,
15};
    private static final byte[] HEX = new byte[]{
        48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 97, 98, 99, 100, 101, 102
    };
    private static final char[] hex = {
        48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 97, 98, 99, 100, 101, 102
    };

    public HexUtils() {
    }

    /**
     * 字符获取十进制
     *
     * @param index
     * @return
     */
    public static int getDec(int index) {
        try {
            return DEC[index - 48];
```

```
    } catch (ArrayIndexOutOfBoundsException var2) {
        return -1;
    }
}
public static byte[] fromHexString(String input) {
    if (input == null) {
        return null;
    } else if ((input.length() & 1) == 1) {
        // 肯定是偶数位，奇数位转换失败：一个 byte 对应两个字符
        throw new IllegalArgumentException("hexUtils.fromHex.oddDigits");
    } else {
        char[] inputChars = input.toCharArray();
        byte[] result = new byte[input.length() >> 1];
        for (int i = 0; i < result.length; ++i) {
            int upperNibble = getDec(inputChars[2 * i]);
            int lowerNibble = getDec(inputChars[2 * i + 1]);
            if (upperNibble < 0 || lowerNibble < 0) {
                // 字符不存在
                throw new IllegalArgumentException("hexUtils.fromHex.nonHex");
            }

            result[i] = (byte) ((upperNibble << 4) + lowerNibble);
```

```
        }


        return result;

    }

  }

}
```

连接和发送控制命令

```java
public static String oldSendLight(String signalLight) {  1 个用法
    // 将十六进制字符串转换为字节数组
    byte[] commandBytes = HexUtils.fromHexString(signalLight);
    //连接的串口是Com17，请连接这个串口然后在发送数据
    // 打开串口
    //COM17串口是要使用的串口将下面代码修改为COM17
    SerialPort serialPort = SerialPort.getCommPort( portDescriptor: "COM16"); // 修改为使用COM17串口
    serialPort.setComPortParameters( newBaudRate: 9600,  newDataBits: 8, SerialPort.ONE_STOP_BIT, SerialPort.EVEN_PARITY);
    serialPort.setComPortTimeouts(SerialPort.TIMEOUT_NONBLOCKING,  newReadTimeout: 1000,  newWriteTimeout: 0);

    if (serialPort.openPort()) {
        try (OutputStream outputStream = serialPort.getOutputStream()) {
            // 发送命令字节数组
            outputStream.write(commandBytes);
            outputStream.flush();
            // 返回操作结果字符串
            return getCommandResult(signalLight);
        } catch (IOException e) {
            e.printStackTrace();
            return "error";
        } finally {
            // 关闭串口
            serialPort.closePort();
        }
    } else {
        return "error";
    }
}
```

改代码片段主要·功能是连接 usb 串口，并将 16 进制字符串命令转为字节发送给灯，从而控制灯的状态

代码：

public static String oldSendLight(String signalLight) {

```
    // 将十六进制字符串转换为字节数组
    byte[] commandBytes = HexUtils.fromHexString(signalLight);
    //连接的串口是 Com17，请连接这个串口然后在发送数据
    // 打开串口
    //COM17 串口是要使用的串口将下面代码修改为 COM17
    SerialPort serialPort = SerialPort.getCommPort("COM16"); // 修改为使用
COM17 串口
    serialPort.setComPortParameters(9600, 8, SerialPort.ONE_STOP_BIT,
SerialPort.EVEN_PARITY);
    serialPort.setComPortTimeouts(SerialPort.TIMEOUT_NONBLOCKING, 1000, 0);


    if (serialPort.openPort()) {
        try (OutputStream outputStream = serialPort.getOutputStream()) {
            // 发送命令字节数组
            outputStream.write(commandBytes);
            outputStream.flush();
            // 返回操作结果字符串
            return getCommandResult(signalLight);
        } catch (IOException e) {
            e.printStackTrace();
            return "error";
        } finally {
```

```
            // 关闭串口
            serialPort.closePort();
        }
    } else {
        return "error";
    }
}
```

完整可测试代码

```
package org.example;

import com.fazecast.jSerialComm.*;
import java.io.IOException;
import java.io.OutputStream;

/**
 * 灯光控制器类，用于通过串口发送控制命令
 */
public class LightController {

    // 预定义的十六进制命令数组（字符串形式）
    public static final String COMMAND_CLOSE_ALL = "010500000000CDCA"; //
```

关闭所有灯光

```
    public static final String COMMAND_GREEN = "01050003FF007C3A";    // 绿灯亮
    public static final String COMMAND_YELLOW = "0105000200006C0A";    // 黄灯亮
    public static final String COMMAND_RED = "01050001FF00DDFA";      // 红灯亮


    public static void main(String[] args) {
        oldSendLight(COMMAND_CLOSE_ALL);
    }


    /**
     * 旧版本发送灯光信号的核心方法
     * @param signalLight 需要发送的灯光命令数组
     * @return 操作结果字符串
   ("close"/"red"/"green"/"yellow"/"unknown"/"error")
     */
    public static String oldSendLight(String signalLight) {
        // 将十六进制字符串转换为字节数组
        byte[] commandBytes = HexUtils.fromHexString(signalLight);
        //连接的串口是 Com17，请连接这个串口然后在发送数据
```

```
        // 打开串口
        //COM17 串口是要使用的串口将下面代码修改为 COM17
        SerialPort serialPort = SerialPort.getCommPort("COM16"); // 修改为使用
COM17 串口
        serialPort.setComPortParameters(9600, 8, SerialPort.ONE_STOP_BIT,
SerialPort.EVEN_PARITY);
        serialPort.setComPortTimeouts(SerialPort.TIMEOUT_NONBLOCKING, 1000,
0);


        if (serialPort.openPort()) {
            try (OutputStream outputStream = serialPort.getOutputStream()) {
                // 发送命令字节数组
                outputStream.write(commandBytes);
                outputStream.flush();
                // 返回操作结果字符串
                return getCommandResult(signalLight);
            } catch (IOException e) {
                e.printStackTrace();
                return "error";
            } finally {
                // 关闭串口
                serialPort.closePort();
```

```
        }
    } else {
        return "error";
    }
}


/**
 * 根据命令字符串返回操作结果字符串
 * @param signalLight 命令字符串
 * @return 操作结果字符串
 */
private static String getCommandResult(String signalLight) {
    switch (signalLight) {
        case COMMAND_CLOSE_ALL:
            return "close";
        case COMMAND_GREEN:
            return "green";
        case COMMAND_YELLOW:
            return "yellow";
        case COMMAND_RED:
            return "red";
        default:
```

```
            return "unknown";
        }
    }

    /**
     * 转换十六进制字符串为字节数组
     */
    public static final class HexUtils {
        private static final int[] DEC = new int[]{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, -1, -1, -1, -
1, -1,
                -1, -1, 10, 11, 12, 13, 14, 15, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -
1, -1,
                -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, 10, 11, 12, 13,
14, 15};
        private static final byte[] HEX = new byte[]{
                48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 97, 98, 99, 100, 101, 102
        };
        private static final char[] hex = {
                48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 97, 98, 99, 100, 101, 102
        };

        public HexUtils() {
```

```
}


/**
 * 字符获取十进制
 *
 * @param index
 * @return
 */
public static int getDec(int index) {
    try {
        return DEC[index - 48];
    } catch (ArrayIndexOutOfBoundsException var2) {
        return -1;
    }
}
public static byte[] fromHexString(String input) {
    if (input == null) {
        return null;
    } else if ((input.length() & 1) == 1) {
        // 肯定是偶数位，奇数位转换失败：一个 byte 对应两个字符
        throw new IllegalArgumentException("hexUtils.fromHex.oddDigits");
    } else {
```

```
            char[] inputChars = input.toCharArray();
            byte[] result = new byte[input.length() >> 1];
            for (int i = 0; i < result.length; ++i) {
                int upperNibble = getDec(inputChars[2 * i]);
                int lowerNibble = getDec(inputChars[2 * i + 1]);
                if (upperNibble < 0 || lowerNibble < 0) {
                    // 字符不存在
                    throw new
IllegalArgumentException("hexUtils.fromHex.nonHex");
                }

                result[i] = (byte) ((upperNibble << 4) + lowerNibble);
            }

            return result;
        }
    }
}
```

编程注意点：

1.发送的命令在转为字节的时候不能带有空格

2.需要将命令转为字节，无法识别字符串命令

3.编程测试时需将其它打开此串口的工具/软件关闭，不然会报错

4.指令发送间隔要求大于 50ms，不然会造成指令失效

5.无法识别串口可能是没有相应驱动，可联系客服下载